

HEALTH ANALYTICS INGESTION PIPELINE

A Serverless ETL Pipeline using Amazon S3, AWS Lambda, and Amazon DynamoDB

1. Project Overview

1.1 Business Scenario

A healthcare analytics team receives COVID-19 data in CSV format from public health sources. The raw dataset is stored in Amazon S3, where AWS Lambda validates and transforms the records. The cleaned data is then loaded into Amazon DynamoDB for analytics and reporting.

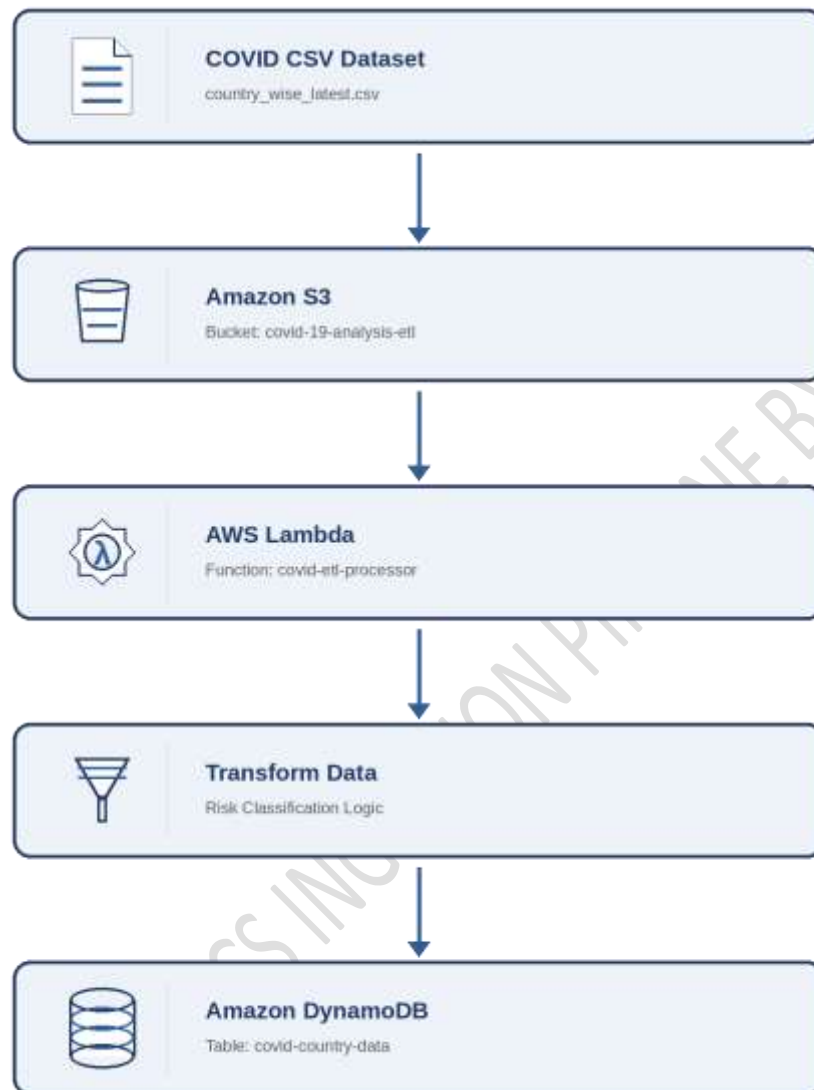
1.2 Dataset & Source

Attribute	Detail
Dataset	country_wise_latest.csv
Source	Kaggle – Corona Virus Report (imdevskp)

https://www.kaggle.com/datasets/imdevskp/corona-virus-report?select=country_wise_latest.csv

Country/R	Confirmed	Deaths	Recovered	Active	New cases	New death	New recov
Afghanista	36263	1269	25198	9796	106	10	18
Albania	4880	144	2745	1991	117	6	63
Algeria	27973	1163	18837	7973	616	8	749
Andorra	907	52	803	52	10	0	0
Angola	950	41	242	667	18	1	0
Antigua an	86	3	65	18	4	0	5
Argentina	167416	3059	72575	91782	4890	120	2057
Armenia	37390	711	26665	10014	73	6	187
Australia	15303	167	9311	5825	368	6	137
Austria	20558	713	18246	1599	86	1	37
Azerbaijan	30446	423	23242	6781	396	6	558
Bahamas	382	11	91	280	40	0	0
Bahrain	39482	141	36110	3231	351	1	421
Banglades	226225	2965	125683	97577	2772	37	1801
Barbados	110	7	94	9	0	0	0

2. AWS Architecture Design



2.1 Architecture Layers

Layer	AWS Service
Source	country_wise_latest.csv
Storage Layer	Amazon S3
Processing Layer	AWS Lambda
Database Layer	Amazon DynamoDB

3. ETL Flow

3.1 Extract

- Read CSV from S3

3.2 Transform

1. Rename Country/Region → country
2. Rename WHO Region → who_region
3. Create risk_status column
4. Handle missing values

3.3 Load

- Store transformed records into DynamoDB

4. Implementation Details

4.1 Amazon DynamoDB Table Creation

Amazon DynamoDB was used as the target NoSQL database for storing transformed COVID-19 records. A table named covid-country-data was created with country as the partition key. The table stores country-wise COVID statistics after data validation and transformation.

Table Detail	Value
Table Name	covid-country-data
Partition Key	country (String)
Database Type	NoSQL (Amazon DynamoDB)

The DynamoDB table acts as the final storage layer in the ETL pipeline, where cleaned and transformed records are loaded for analytics and reporting.

4.2 AWS Lambda Function Creation

AWS Lambda was used as the processing layer of the ETL pipeline. A serverless function named covid-etl-processor was created using the Python 3.12 runtime. The Lambda function is responsible for:

- Reading the COVID-19 CSV dataset from Amazon S3
- Performing data validation and transformation
- Generating a risk classification for each country
- Loading transformed records into Amazon DynamoDB

Using AWS Lambda eliminates the need to manage servers and enables automatic execution of ETL workloads.

4.3 Lambda Environment Configuration

Environment variables were configured in the AWS Lambda function to make the ETL pipeline reusable and configurable.

Variable	Value
S3_BUCKET	covid-19-analysis-etl
S3_KEY	covid-data/country_wise_latest.csv
DYNAMODB_TABLE	covid-country-data

Using environment variables avoids hardcoding resource names inside the source code and improves maintainability.

4.4 ETL Transformations Implemented

The Lambda function performs the following transformations:

- Renamed source columns into business-friendly attributes
- Converted numeric values into integer format
- Generated a derived field called risk_status
- Classified countries into High Risk, Medium Risk, and Low Risk categories based on confirmed cases

Transformation Logic

Condition	Risk Status
Confirmed Cases > 100,000	High Risk
Confirmed Cases > 10,000	Medium Risk
Otherwise	Low Risk

These transformations improve data quality and make the dataset more suitable for analytics.

4.5 Lambda Function Code

```
import csv
import io
import json
import os
from decimal import Decimal
import boto3

S3_BUCKET = os.environ["S3_BUCKET"]
S3_KEY = os.environ["S3_KEY"]
DYNAMODB_TABLE = os.environ["DYNAMODB_TABLE"]

def lambda_handler(event, context):
    s3 = boto3.client("s3")
    dynamodb = boto3.resource("dynamodb")
    table = dynamodb.Table(DYNAMODB_TABLE)
```

```

response = s3.get_object(
    Bucket=S3_BUCKET,
    Key=S3_KEY
)
csv_content = response["Body"].read().decode("utf-8")
reader = csv.DictReader(io.StringIO(csv_content))
records_written = 0

with table.batch_writer() as batch:
    for row in reader:
        country = row["Country/Region"]
        confirmed = int(float(row["Confirmed"]))
        deaths = int(float(row["Deaths"]))
        recovered = int(float(row["Recovered"]))
        active = int(float(row["Active"]))
        who_region = row["WHO Region"]

        # Transformation
        if confirmed > 100000:
            risk_status = "High Risk"
        elif confirmed > 10000:
            risk_status = "Medium Risk"
        else:
            risk_status = "Low Risk"

        item = {
            "country": country,
            "confirmed": confirmed,
            "deaths": deaths,
            "recovered": recovered,
            "active": active,
            "who_region": who_region,
            "risk_status": risk_status
        }

        batch.put_item(Item=item)
        records_written += 1

return {
    "statusCode": 200,
    "body": json.dumps({
        "message": "COVID ETL completed successfully",
        "records_written": records_written
    })
}

```

5. IAM Permissions Configuration

AWS Identity and Access Management (IAM) policies were configured to provide secure access to AWS resources. The Lambda execution role was granted the following permissions:

Service	Permission	Purpose
Amazon S3	s3:GetObject	Allows Lambda to read the COVID-19 CSV dataset stored in the S3 bucket
Amazon DynamoDB	dynamodb:PutItem	Allows Lambda to insert transformed records into the DynamoDB table
Amazon DynamoDB	dynamodb:BatchWriteItem	Allows Lambda to insert transformed records into the DynamoDB table

The principle of least privilege was followed by granting only the permissions required for ETL processing.

Troubleshooting Note

During testing, the Lambda function initially failed due to the default execution timeout of 3 seconds. The timeout was increased to 30 seconds and memory allocation was increased to 256 MB to support processing of larger CSV datasets. After the configuration update, the ETL pipeline executed successfully.

6. ETL Execution Results

The ETL pipeline executed successfully using AWS Lambda. The COVID-19 dataset stored in Amazon S3 was processed and loaded into Amazon DynamoDB.

Metric	Value
Source Dataset	country_wise_latest.csv
Storage Layer	Amazon S3
Processing Layer	AWS Lambda
Target Database	Amazon DynamoDB
Records Processed	187
Records Loaded	187

The Lambda function successfully validated, transformed, and inserted all records into the DynamoDB table. The transformed dataset included a derived attribute called `risk_status`, which classified countries based on confirmed COVID-19 cases. The successful execution demonstrated the implementation of a serverless ETL pipeline using AWS services.

7. Conclusion

A serverless ETL pipeline was successfully developed using AWS services. The COVID-19 dataset was stored in Amazon S3, processed using AWS Lambda, transformed according to business rules, and loaded into Amazon DynamoDB.

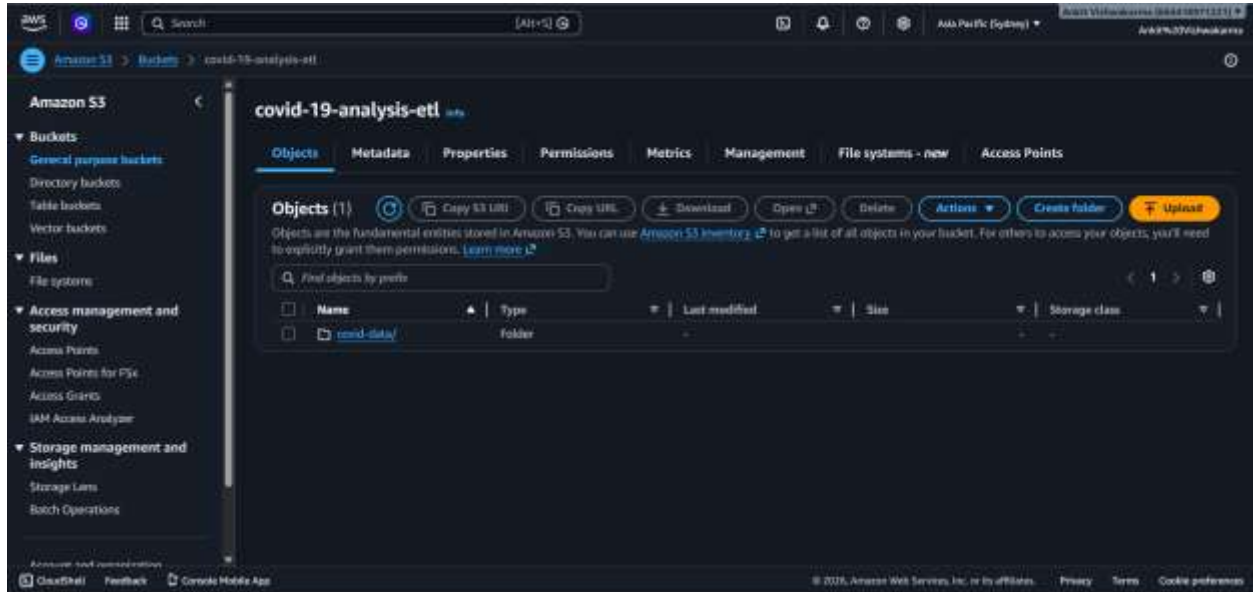
The solution demonstrated the practical implementation of Extract, Transform, and Load (ETL) concepts using cloud-native services. A total of 187 records were processed and inserted into DynamoDB. Additional business logic was implemented by classifying countries into risk categories based on confirmed COVID-19 cases.

This project highlighted the advantages of serverless architectures, including scalability, reduced operational overhead, and cost efficiency.

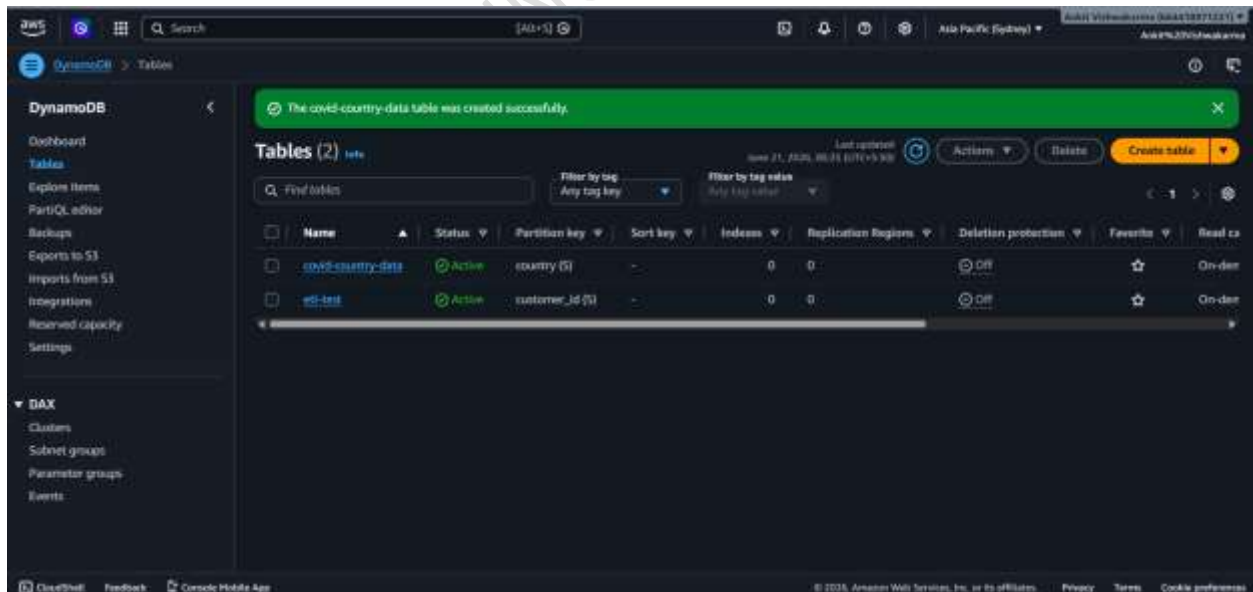
HEALTH ANALYTICS INGESTION PIPELINE BY AAKIT

7. Screenshots

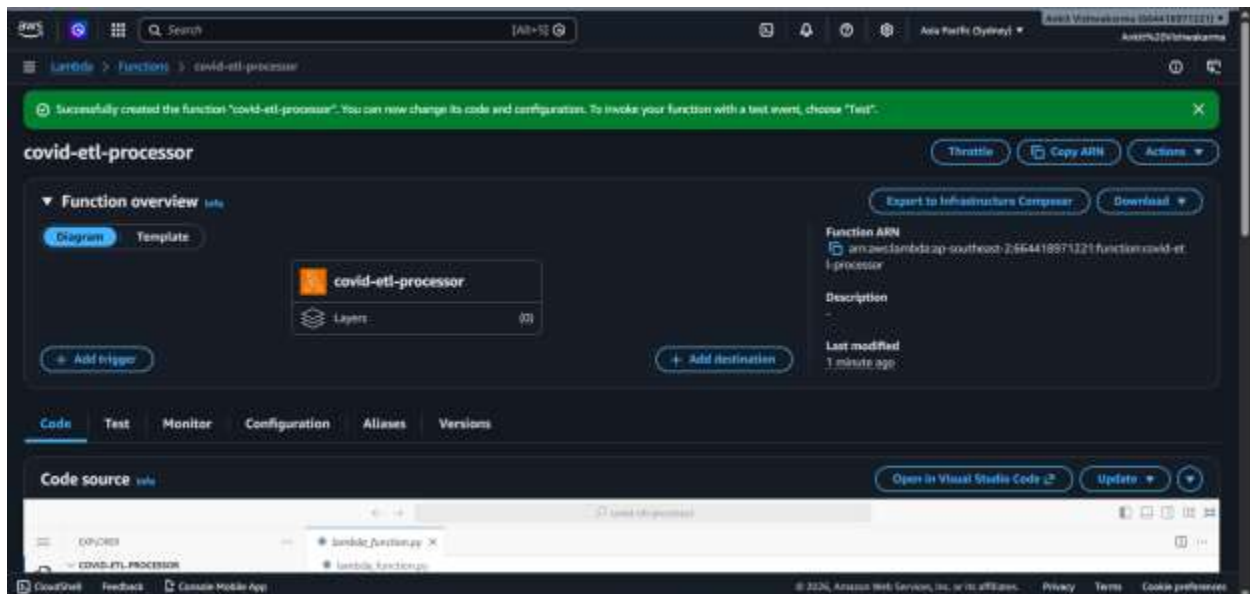
7.1 S3 Bucket with CSV file



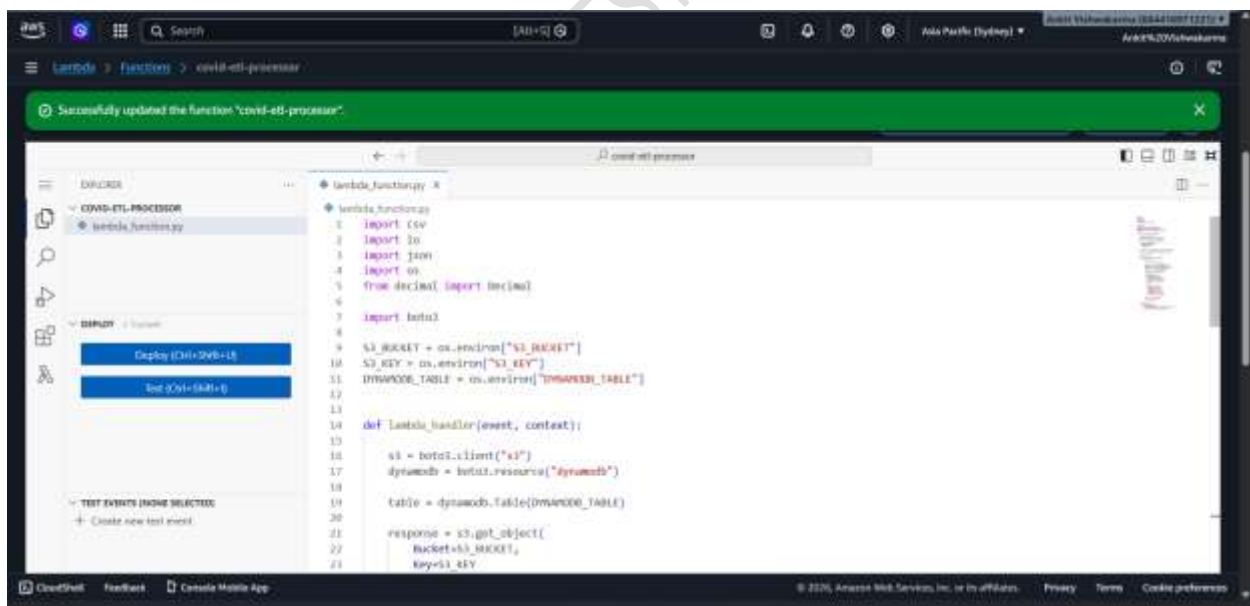
7.2 DynamoDB Table Creation



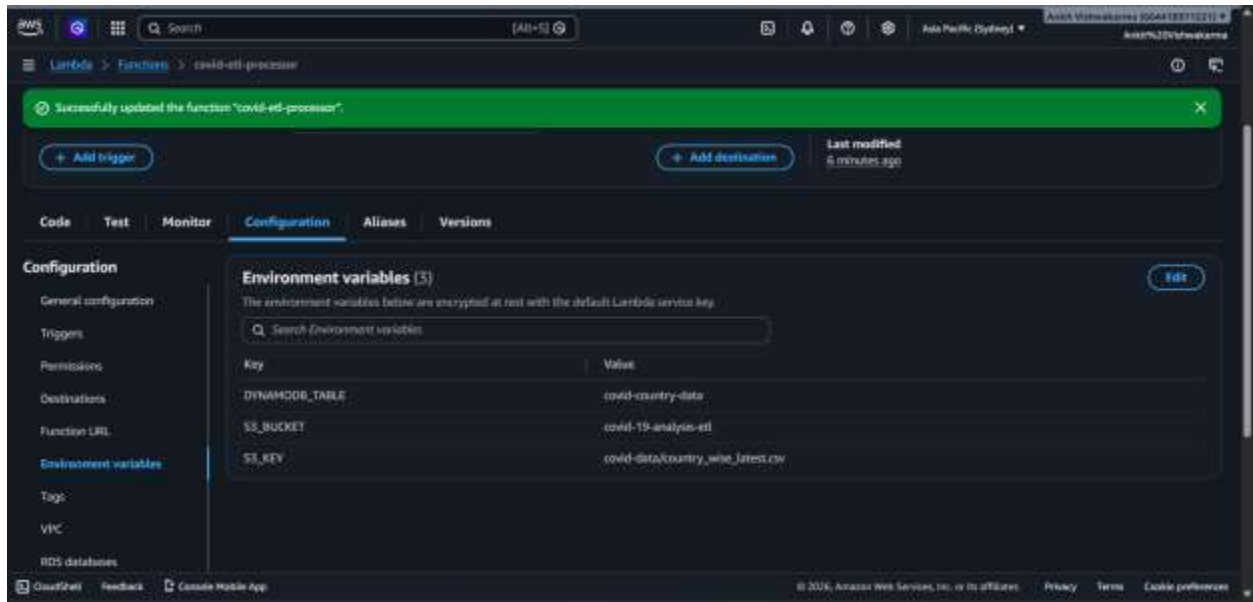
7.3 Lambda Function



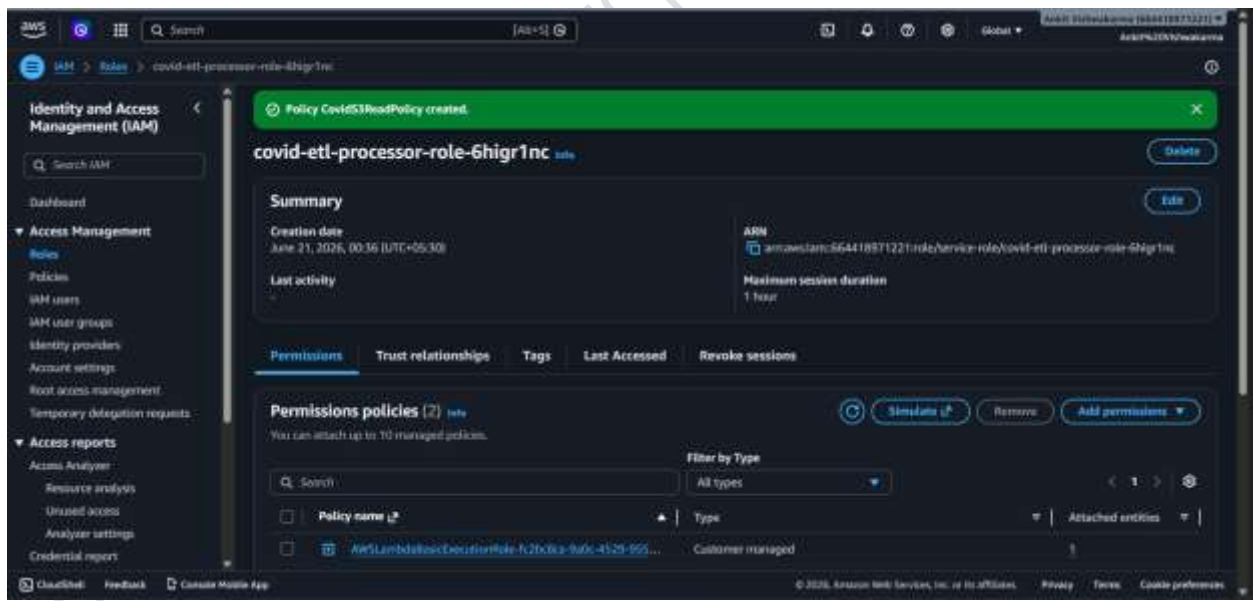
7.4 Lambda Function Code



7.5 Lambda Function Environment Variables



7.6.1 IAM Policies (S3 Policy)



7.6.2 IAM Policies (DynamoDB Policy)

The screenshot shows the AWS IAM console with a green notification banner at the top stating "Policy CovidDynamoWritePolicy created." Below this, the policy details are displayed: Creation date (June 21, 2026, 00:36 (UTC+05:30)), Last activity, and Maximum session duration (1 hour). The policy is associated with the role "arn:aws:iam::564418971221:role/service-role/covid-eti-processor-role-0tag1nc".

The "Permissions policies (3)" section shows a list of policies attached to the role:

Policy name	Type	Attached entities
AWSLambdaBasicExecutionRoleFC2bRz9aX-4529-955-	Customer managed	1
CovidDynamoWritePolicy	Customer inline	0
CovidS3ReadPolicy	Customer inline	0

7.7 DynamoDB Records

The screenshot shows the AWS DynamoDB console with the "covid-country-data" table selected. The table has 50 items returned. The scan started on June 21, 2026, 01:14:21.

country (String)	active	confirmed	deaths	recovered	risk_status
Georgia	199	1137	16	923	Low Risk
US	2816444	4290259	148011	1325904	High Risk
Senegal	3089	9764	194	6477	Low Risk
Equatorial Guinea	2176	3071	51	842	Low Risk
Guinea-Bissau	1125	1054	26	803	Low Risk
Argentina	91782	167416	3659	72575	High Risk
Angola	667	950	41	242	Low Risk
Netherlands	47064	53413	6160	189	Medium Risk
Niger	36	1132	69	1027	Low Risk
Greenland	1	14	0	13	Low Risk

8. Reflection Questions

1. Why is S3 a good landing zone for raw files?

Amazon S3 is highly scalable, durable, and cost-effective. It is ideal for storing raw data before processing.

2. Why is Lambda suitable for this small ETL workload?

AWS Lambda is serverless, automatically scales, and charges only for execution time, making it cost-efficient for small ETL jobs.

3. Why did you choose your DynamoDB partition key?

I chose **country** as the partition key because it uniquely identifies each record and enables fast lookups.

4. What is the difference between storing this data in DynamoDB vs RDS?

- **DynamoDB:** NoSQL, schema-flexible, highly scalable, low-latency access.
- **RDS:** Relational database, supports SQL, fixed schema, better for complex joins and transactions.

5. Why should we avoid scanning large DynamoDB tables in production?

Scanning reads every item in the table, which increases latency, consumes more read capacity, and raises costs.

6. What IAM permissions did your Lambda function need?

- **S3:** `s3:GetObject` (to read the CSV file)
- **DynamoDB:** `dynamodb:PutItem`, `dynamodb:BatchWriteItem` (to insert records)
- **CloudWatch Logs:** Basic Lambda execution permissions for logging.